

Modernizing Legacy Apps:

MICRO-FRONTEND ARCHITECTURE, AND CORE-AGNOSTIC DESIGN



Daniel David Díaz González – Software Engineer
Head of Frontend at Yanovis

What We'll Explore Today

- 🏗️ Micro-frontend Architecture
- 🧩 Core-Agnostic Project
- 📦 ImportMaps and Systemjs for dynamic loading.
- ⚙️ Rsbuid vs Vite: choosing the right build tool
- 🚀 Progressive migration from Vue2 to Vue3

💡 *All based on a real migration from a legacy monolith to a micro-frontend ecosystem.*



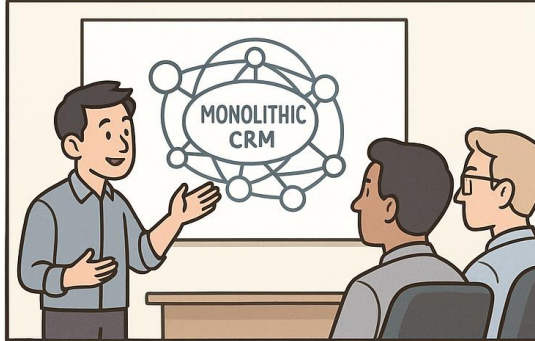
Hello

The illustration features a man with brown hair and a friendly expression, wearing a teal t-shirt, sitting at a desk with a laptop. He is waving his right hand. The background is a gradient of blue and green, filled with faint, stylized code symbols such as curly braces, less-than and greater-than signs, and equals signs. To the right of the man is a blue square with the white letters 'TS'. In the bottom right corner, there is a small potted plant with green leaves. In the bottom left corner, a portion of a purple mug is visible. The overall style is clean and modern, typical of digital marketing or educational content.

TS



Meet Nico: a passionate frontend developer, always **eager to learn**



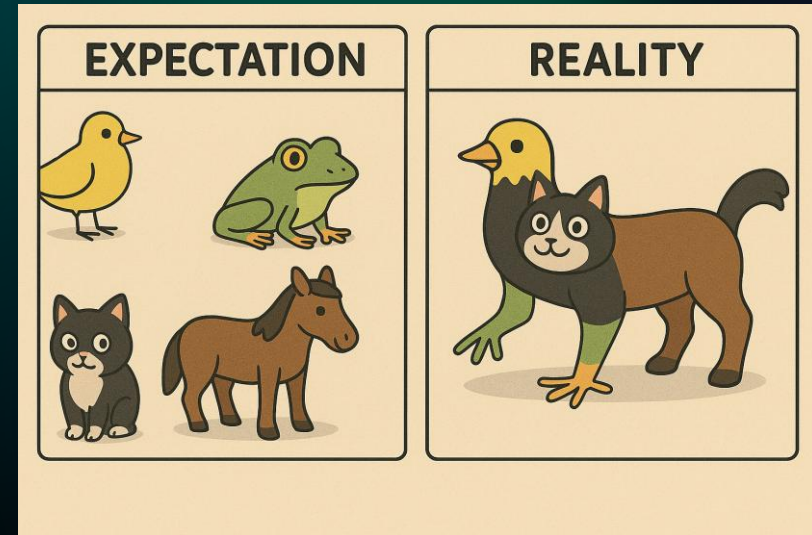
At his company, a CRM has grown into a tangled monolith...



Every fix breaks something else.



It's chaos.



One day, his boss tells him that they want to start migrating to Vue 3, start using TypeScript, and use modern libraries and frameworks that aren't compatible with Vue 2.



The idea sounds great, but Nico knows there's a big challenge:

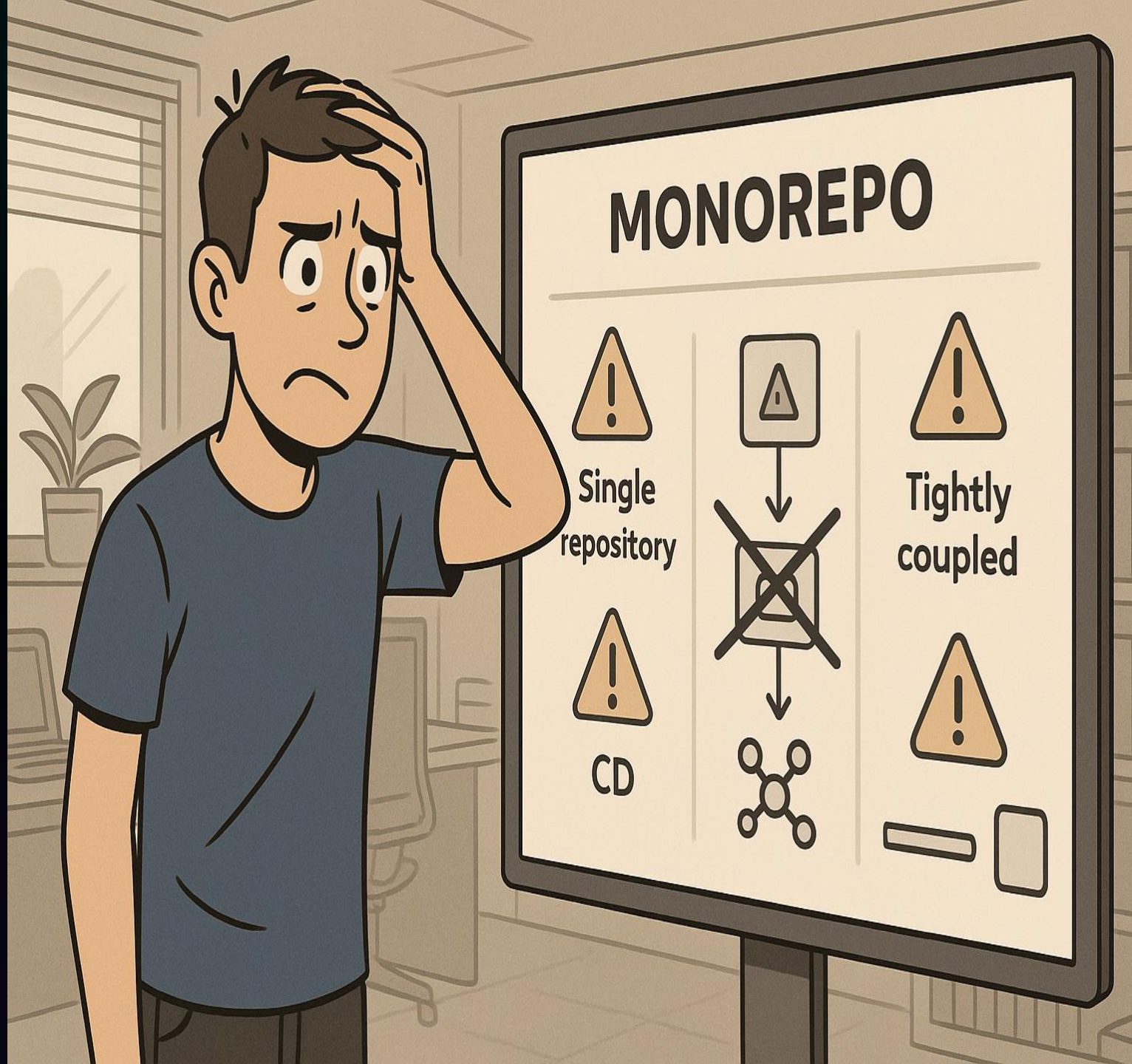


And they can't migrate everything at once

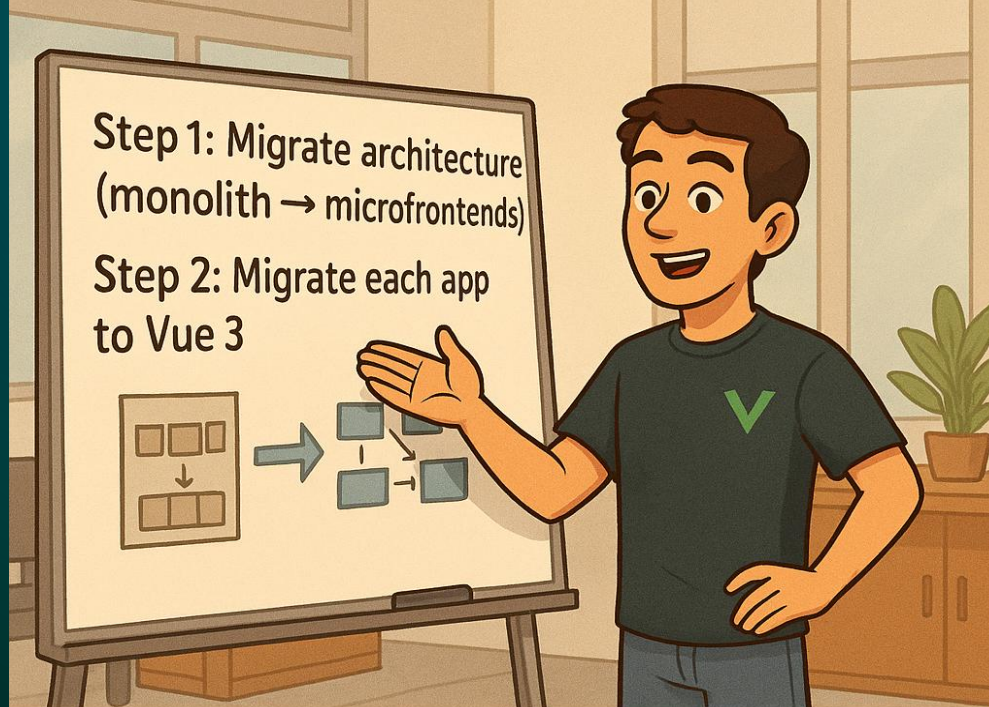
They can't stop developing new features for months.



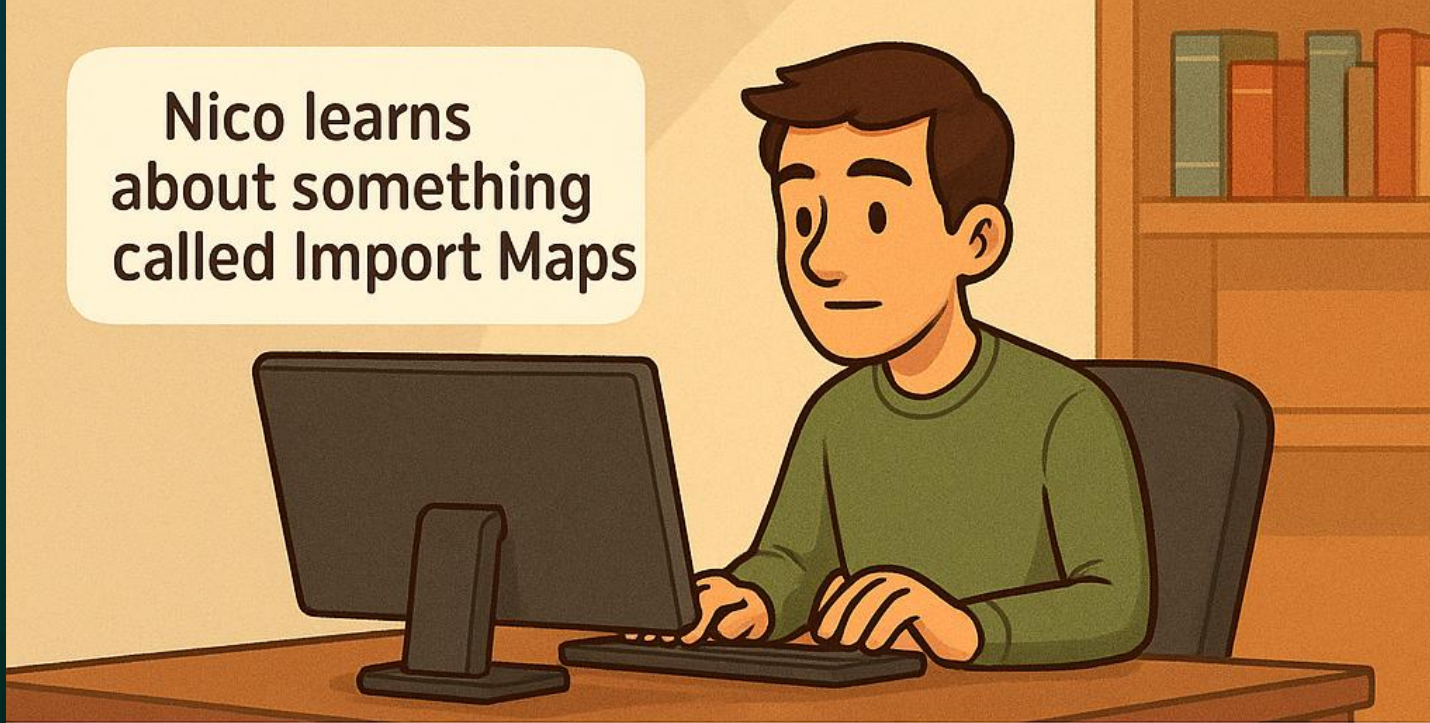
And they can't migrate everything at once







Nico learns
about something
called Import Maps



A browser feature
that allows
controlling how
JavaScript module
imports are resolved
without a tool
like Webpack



But there are dependencies
not compatible with Vue 3,
many tightly coupled to
the framework.



Authentication

WebSockets

Internationaliza-
tion (i18n)

Global Reactivity

Sentry (for error
monitoring)

HTTP Fetch Data

Create a core agnostic to
the framework, rewriting
all that cross-cutting logic
in pure TypeScript.



Done

- RxJS for reactivity & global states
- i18next (vanilla) to handle i18n
- Sentry SDK in TypeScript
- Fetch and auth tools decoupled from framework

Vite is fast, but
no SystemJS.
Webpack is slow.



Vite



Webpack

Rsbuild — Rust-based,
fast, supports SystemJS!



Rsbuild

Modern Frontend Architecture



Orchestrator App

Controls dynamic loading of microfrontends



Agnostic Core

Shared functionality in pure TypeScript



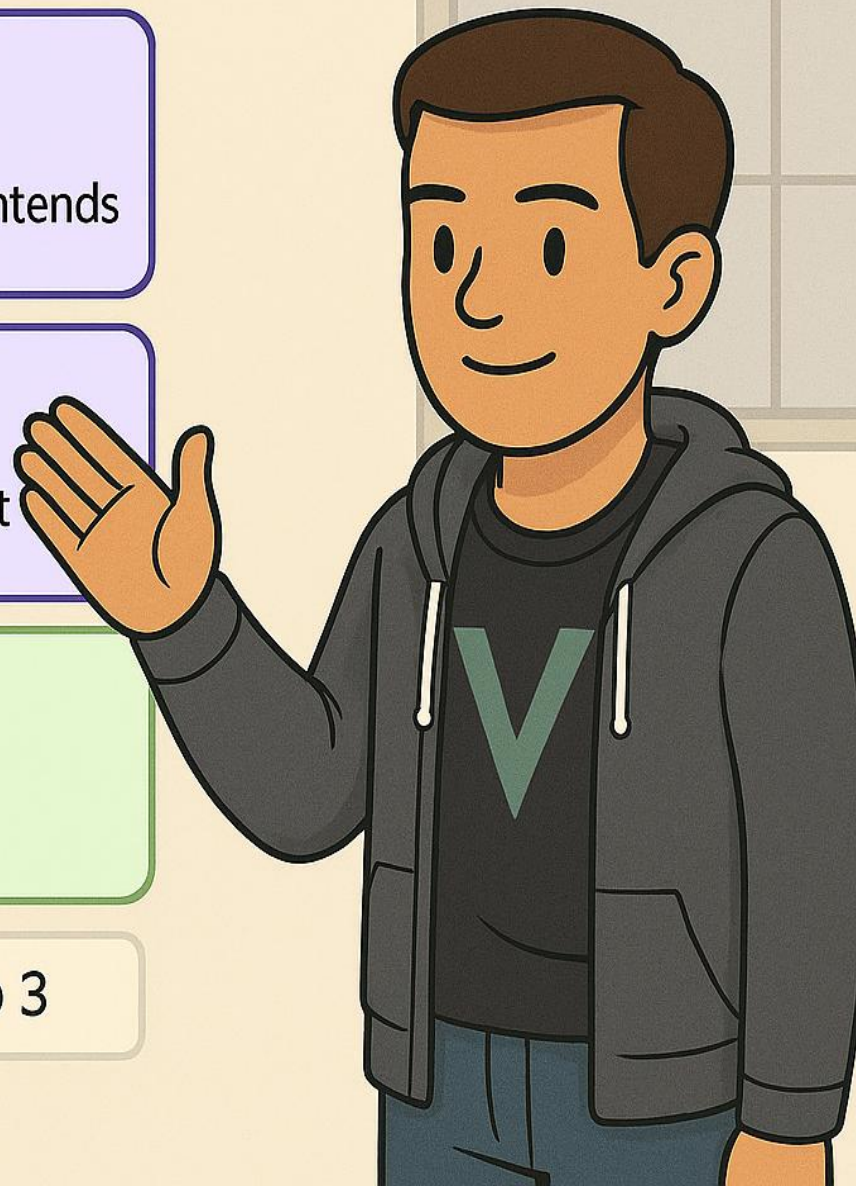
Shared Components

Reusable UI components

App 1

App 2

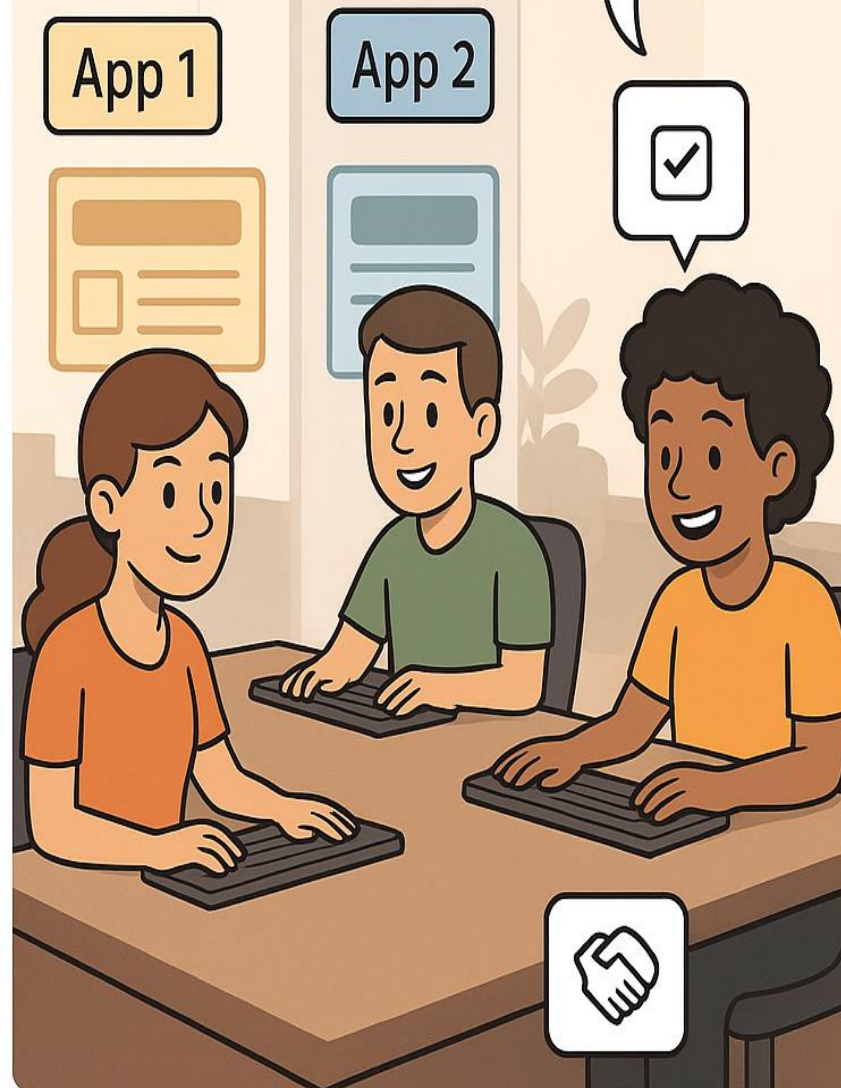
App 3



With this architecture,
the apps are totally
independent.



And the teams can
work in parallel.





Thanks!

I'll now hand
things over to
my colleague
David.

Override Apps

The screenshot shows the Yanovis Travel Suite web application. The top navigation bar includes the Yanovis logo, a user profile for David (Hotel Luxurious... (263)), and several notification icons. The main content area is titled 'Scegli l'applicazione' (Choose the application) and features a 'Selezione dell'app' (App selection) section with four app cards: Communication App, Payment App, Newsletter App, and Profiling App. Below this is the 'Import Map Manager' section, which allows users to view and override import maps. It includes buttons for 'Add new module', 'Edit Import Map', 'Disable all overrides', and 'Remove all overrides'. A search bar is present, and a table lists modules and their domains.

Module Name	Domain
@yanovis/v3-shared-components	http://localhost:8821/js/app.js https://testing-travelsuite.yanovis.com/apps/v3-shared-components/20250522113834_18ddb09/js/app.js
vue	https://cdn.yanovis.com/js/npm/vue@2.7.14/dist/vue.min.js
rxjs	https://cdn.yanovis.com/js/npm/rxjs@7.8.2/dist/bundles/rxjs.umd.min.js
vuex	https://cdn.yanovis.com/js/npm/vuex@3.6.2/dist/vuex.min.js
axios	https://cdn.yanovis.com/js/npm/axios@0.22.0/dist/axios.min.js
dayjs	https://cdn.yanovis.com/js/npm/dayjs@1.11.13/dayjs.min.js

ImportMaps

It simplify Javascript module management

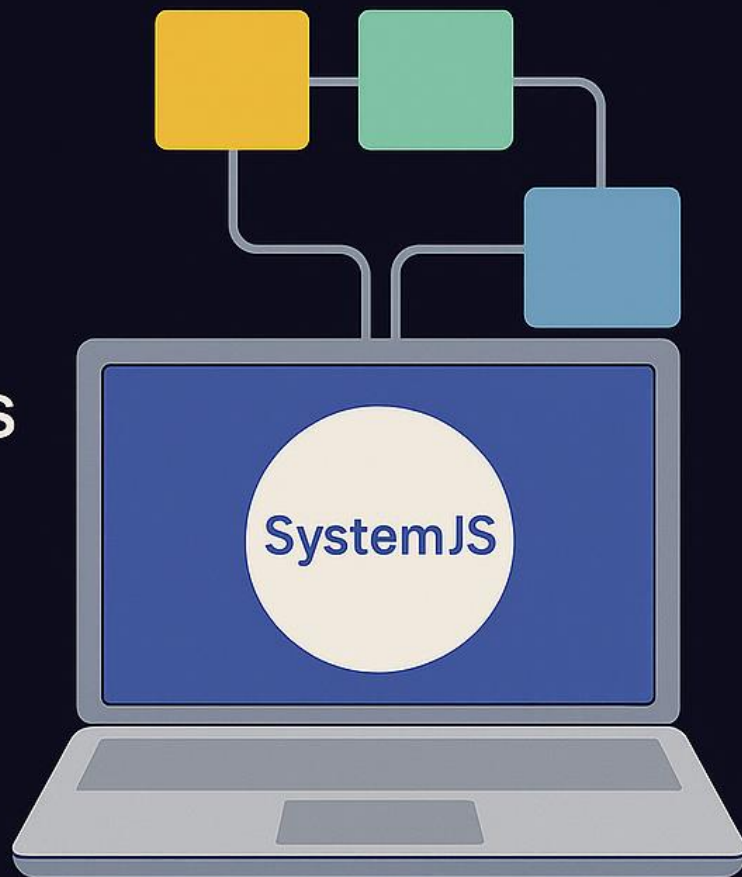
```
{
  "imports": {
    "@yanovis/app-navbar": "///localhost:8801/js/app.js",
    "@yanovis/app-admin": "///localhost:8802/js/app.js",
    "@yanovis/app-pay": "///localhost:8803/js/app.js",
    "@yanovis/app-ticketing": "///localhost:8804/js/app.js",
    "vue": "https://cdn.yanovis.com/js/npm/vue@2.7.14/dist/vue.min.js",
    "vuetify": "https://cdn.yanovis.com/vuetify@2.6.16/dist/vuetify.min.js"
  },
  "scopes": {
    "///localhost:8804/": {
      "vue": "https://cdn.yanovis.com/js/npm/vue@3.5.13/dist/vue.min.js",
      "vuetify": "https://cdn.yanovis.com/vuetify@3.8.1/dist/vuetify.min.js"
    }
  }
}
```

Webpack, RsBuild, Vitejs, etc

```
external:['vue','vuetify']
```

What is SystemJS?

- Module loader library for the browser
- Supports loading ES modules and other formats
- Useful for microfrontend architecture



Route Orchestrator Project with Single SPA

```
<single-spa-router>

  <application name="@yanovis/app-navbar"/>

  <route path="admin">
    <application loader="loader" name="@yanovis/app-admin"/>
  </route>

  <route path="pay">
    <application loader="loader" name="@yanovis/app-pay"/>
  </route>

  <route path="ticketing">
    <application loader="loader" name="@yanovis/app-ticketing"/>
  </route>

</single-spa-router>
```

Agnostic Project Idea

Global Project

```
export class AuthService {  
  private userSubject = new BehaviorSubject(loadUserFromStorage());  
  
  readonly user$ = this.userSubject.asObservable();  
  
  setUser(user: User | null) {  
    this.userSubject.next(user);  
  }  
}
```

Apps Consuming

```
AuthService.user$.subscribe(user => {  
  handleAuthChange(user);  
});  
  
// Or via a Vue composable  
const user = useObservable(AuthService.user$);
```

Island Architecture vs Microfrontends

Island Architecture



Astro renders various frameworks, but it's still one build and one ownership.

Microfrontends



Each microfrontend has a separate ownership and build.

When not to use it

- **Too Complex for Small Apps**

Micro-Frontends may be overkill for simpler applications—monoliths often work fine.

- **Not good for small teams**

Handling shared state and team coordination can be too much.

- **Steep Learning Curve**

Inexperienced teams may face issues, leading to technical debt.



/ Thanks ;

